BundleFuse, Component by Component

How every piece is built—from the raw data to the final score—with no hand-waving

Study note

Abstract

This note explains *how* BundleFuse *actually works*, one component at a time, in plain language. For every piece we answer three questions in order: **(i) what is it in general** (the textbook idea), **(ii) how is it defined *here*** (the exact quantities this code plugs into that idea—because "PMI" and "SVD" mean nothing until you say *PMI of what* and *SVD of which matrix*), and **(iii) a worked numerical example** on *one real* POG *query* carried through the whole note (a tiny 4-item sketch is used only where a full matrix must be printed to show its shape). Where the method consumes pretrained **embedding vectors**, we say so explicitly and give their exact dimensions and origin. Nothing here is guessed: every formula is read off `bundlefuse/operators.py` and `bundlefuse/eval/fullrank.py`, and every number in the running example is machine-verified. A closing section (§8) then runs the *entire* scorer on one **real POG** query—actual catalogue items, actual value for every signal—so the whole pipeline is seen filling a genuine partial bundle end to end.

Contents

1 How to read this note **2**

2 What we start with: the raw inputs **3**
2.1 The items, the bundles, and the incidence matrix 3
2.2 The embedding features — stated explicitly 4
2.3 The side labels: categories, artists, albums 5

3 Three structures we derive immediately **6**

4 The vocabulary of operations (general idea → local definition → example) **7**
4.1 Co-occurrence scoring: "who shows up with the seeds" 7
4.2 PMI / PPMI: "is this pair together *more than chance*?" 7
4.3 Truncated SVD: a dense "collaborative fingerprint" per item 9
4.4 Degree normalisation: taming hubs and big bundles 10
4.5 Graph propagation: a two-hop smoothing walk 10

5 The signals, built one by one **11**
5.1 Collaborative family (from B only — no embeddings) 11
5.2 Taxonomic family (from category/artist/album labels) 12
5.3 Content family (from the embedding vectors) 13

6 From a signal to per-query numbers: scoring and standardising **15**

7 Fusion: four stages to the final score **15**
7.1 Why validate at NDCG@20 — and why not a "cutoff-free" loss 18

8 The same machine on a real POG query **19**

9 The ledger: what is learned vs. fixed **20**

1 How to read this note

BundleFuse solves **bundle completion**: you are shown a few items that belong together (the *seeds* S—part of an outfit, a playlist, a shopping basket) and you must rank the rest of the catalogue so the genuine missing members come out on top. The method does this by computing many cheap "opinions" about each candidate item—co-occurrence, taxonomy, content similarity—**standardising** them onto a common scale, **blending** them with a handful of weights picked on a validation set, and finally **routing** the blend against a cold-start fallback so that rarely-seen items are still rankable. *The scoring core trains no neural network and has zero learned weights*; the only numbers "learned" are a few dials chosen by grid search on validation data. Embedding vectors *are* used, but they are **fixed inputs** shipped with the datasets, not things we train (§2). Figure 1 is the whole method on one line—a partial bundle in, a completed bundle out—and every later section just explains one of its boxes.

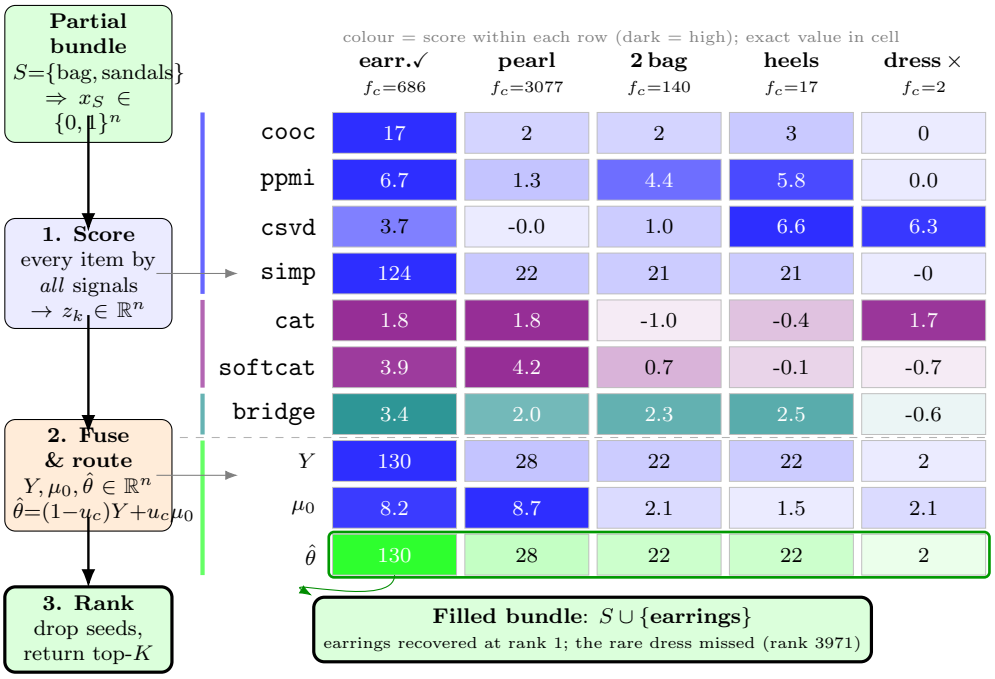


Figure 1. How a partial bundle is filled: the flow (left) aligned to the real score map (right). The query is the real POG outfit $S = \{\text{bag, sandals}\}$; columns are five candidates (two held-out: the **earrings** ✓ and the **dress** ×), rows are the signals, grouped by family, then the fusion scores $Y, \mu_0, \hat{\theta}$. **(1) Score:** every signal turns x_S into one value per item (cells; shaded dark where high *within that row*, exact value inside)—collaborative `cooc/simp` and taxonomic `cat/softcat` both back the earrings, while `csvd` alone rates the rare dress and heels highest. **(2) Fuse & route:** the rows collapse into Y (collab. blend), μ_0 (prior) and $\hat{\theta} = (1 - u_c)Y + u_c\mu_0$; here κ=0 so $\hat{\theta}$=Y. **(3) Rank:** order $\hat{\theta}$ (green row), drop the seeds, return top-K: the **earrings** come out at rank 1, the rare **dress** is missed (rank 3971). Every number is the live scorer's output (`results/case_study_pog_dense.json`); the signals are worked one by one in §5–§7 and consolidated in Fig. 17.

The note is bottom-up, and it is built as a **story with a problem driving every step**: we start from the simplest possible signal, watch it fail on a concrete case, add exactly the piece that fixes that failure, and repeat. So co-occurrence (§4) is biased toward popular hubs ⇒ PMI corrects for popularity (§4.2); PMI still only sees pairs that *literally* co-occurred ⇒ the SVD fingerprint adds indirect, partners-of-partners evidence (§4.3); all of these need co-occurrence, so a *cold* item with none is invisible to them ⇒ taxonomy and content signals rank it by *what it is* (§5.2–§5.3); the resulting ∼eleven opinions *disagree*, so fusion blends them and routing decides, per candidate, how much to trust co-occurrence versus the metadata prior (§7). Concretely: §2–§3 define the inputs; §4 the recurring *operations*; §5 the individual *signals*; §6 how a signal becomes per-query numbers; §7 the four-stage fusion; §7.1 why we validate at a short cutoff. Then §8 runs the *entire* machine on one **real POG** query—actual items, actual numbers for every signal—so you can watch a partial bundle get filled end to end, and §9 is the ledger of what is learned versus fixed.

The running example (used from here on): one real POG_dense query. The partial outfit is

$$S = \{\ \underbrace{\text{sequined chain shoulder } \textbf{bag}}_{\text{item 4399, cat. "bags"}},\quad \underbrace{\text{block-heel strappy } \textbf{sandals}}_{\text{item 25026, cat. "sandals"}}\ \},$$

drawn from the real catalogue of n = 31,217 items and m = 19,865 training outfits. Two members were held out and must be recovered: a pair of **drop earrings** (item 30318) and a rare **plus-size A-line dress** (item 5716). **Every worked number in this note is the value the live scorer actually computes for this query** (dumped by `CASE_STUDY=1 BUNDLE_DS=pog_dense`; the full per-signal table is §8, Fig. 17). Where a matrix is small enough to print on paper, we also show a **4-item sketch** $\{1, 2, 3, 4\}$ with bundles b_1=$\{1, 2, 3\}$, b_2=$\{1, 2, 4\}$, b_3=$\{2, 3\}$—*purely to see the shape* of B, C and PPMI, since the real ones are 31,217 × 31,217.

2 What we start with: the raw inputs

2.1 The items, the bundles, and the incidence matrix

The **catalogue** is a finite set of items $\mathcal{I} = \{1, \ldots, n\}$. The **training data** is a collection of m bundles $\mathcal{B} = \{b_1, \ldots, b_m\}$, each a subset of items. We record them once in a **bundle–item incidence matrix**

$$B \in \{0, 1\}^{m \times n}, \qquad B_{ji} = \begin{cases} 1 & \text{item } i \text{ is in bundle } j \\ 0 & \text{otherwise.} \end{cases}$$

Every bundle is a *row*; every item is a *column*. This single matrix is the backbone of every collaborative signal. (In code: `SimplicialGraphOperator` builds `self.B` and its binary version `self.B_bin`.)

Example 1 (the incidence matrix — 4-item shape sketch). For the 4-item sketch,

$$B = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} \begin{matrix} \leftarrow b_1 = \{1,2,3\} \\ \leftarrow b_2 = \{1,2,4\} \\ \leftarrow b_3 = \{2,3\} \end{matrix}$$

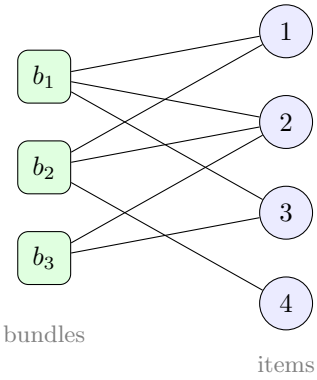


Figure 2. *The bundle–item graph of the 4-item sketch* (shape only; the real graph has 31,217 item nodes). Each edge is a 1 in B. The **degree of an item** (number of edges) is its frequency f: item 2 has 3 edges ($f_2 = 3$), item 4 has one ($f_4 = 1$). Two items **co-occur** ($C_{ci} > 0$) exactly when they connect to a common bundle box: items 1 and 2 share b_1 and b_2 (so $C_{12} = 2$), while items 3 and 4 share no box ($C_{34} = 0$). Every collaborative signal in this note is some way of reading this one picture.

2.2 The embedding features — stated explicitly

Besides the bundles, each item carries **pretrained embedding vectors**: fixed real-valued feature vectors that describe the item's *content* (what it looks like, what it is called, what it sounds like). **BundleFuse does not train these. It loads them from the public dataset release and treats them as constants.** They enter only the *content* and *soft-category* signals (§5.3, §5.2); the collaborative and taxonomic signals never touch them. The exact tensors (read in the feature-loading block of `eval/fullrank.py`) are:

File on disk	Role in code	Shape (e.g. POG)	What it is
`title_emb.npy`	`feats["content"]`	48676×512	text embedding of the item *title*
`content_feature.pt`	`feats["image"]` (fashion)	48676×768	*image* embedding of the product photo
	`feats["audio"]` (music)	254155×512	*audio* embedding of the track
`description_feature.pt`	`feats["desc"]`	48676×768	text embedding of the *description*

Where every vector comes from (so no number is unexplained). There are two distinct sources, and we name both. *(i) Image / description / audio — from the public release.* `content_feature.pt` and `description_feature.pt` are the pre-extracted features shipped with the public BundleConstruction release (Liu et al., *Fine-tuning Multimodal LLMs for Product Bundling / Bundle-MLLM*, KDD 2025) and consumed unchanged — the same features the heavy baselines use. Their origin is documented in the **CLHE** paper (Ma et al., "Leveraging Multimodal Features and Item-level User Feedback for Bundle Construction," WSDM 2024; `arXiv:2310.18770`), whose dataset the BundleConstruction release *is*, and corroborated by RaMen (ECAI 2025, §3.1.2): for POG the visual and textual vectors are produced by **pre-trained**

BLIP (Li et al., "BLIP: Bootstrapping Language-Image Pre-training," ICML 2022), hence their 768 dimensions; for SPOTIFY the audio vector is produced by **CLAP** (Wu et al., "Large-scale Contrastive Language–Audio Pretraining," 2023), which is why Spotify's content vector is 512-d (CLAP's native embedding size) rather than 768-d. The release *also* ships LightGCN collaborative features (`item_cf_feature.pt`, `*_LightGCN_bi_feature.pt`), which we deliberately do **not** load — reusing precomputed collaborative features under our re-split would leak held-out co-occurrence (it once manufactured a 16× phantom win on POG); we rebuild every collaborative signal from the train fold instead. *(ii) Title — generated by us.* `title_emb.npy` (48676×512) is **not** from the release and **not** a mystery: *we* produce it with the script `embed_titles.py`, which runs a frozen `BAAI/bge-small-zh-v1.5` Chinese sentence-encoder (via `sentence_transformers`, `normalize_embeddings=True`) over the POG item titles — which are in Chinese, since POG is Alibaba iFashion. Its 512 dimensions are exactly that model's hidden size; the companion `title_emb_large.npy` (1024-d) is the identical script with `BAAI/bge-large-zh-v1.5`. So every content input is now sourced: BLIP (release, via CLHE) for POG image/description, CLAP (release, via CLHE) for SPOTIFY audio, and a frozen BGE encoder (ours) for the POG titles.
Every vector is L_2**-normalised** on load (`_loadpt`: $x \leftarrow x/\|x\|_2$), so a dot product between two of them is a cosine similarity in $[-1, 1]$. For POG the three are concatenated into a single $512{+}768{+}768 = 2048$-dimensional multimodal vector (`mm_feat`) used for clustering; for music only the 512-d audio vector exists (the code detects this via the *absence* of `title_emb.npy` and sets the internal `MUSIC` flag).

Not every dataset uses an external encoder. The table above is for POG and SPOTIFY only. On the e-commerce sets (ELECTRONIC/FOOD/CLOTHING) and the grocery set (DUNNHUMBY) there is **no pretrained embedding at all**: the content signal is a plain **TF–IDF** vector of the item's own text fields (department / commodity / brand / title), reduced by truncated SVD, computed *inside* the pipeline (`eval/bundlerec.py` and `eval/dunnhumby.py`). So "content" means two different things depending on the dataset — a deep BLIP/CLIP embedding on POG/Spotify, a classical bag-of-words vector on the e-commerce/grocery sets — and we say so plainly rather than imply one uniform source. **If a dataset ships neither, the content signals are simply absent and the method runs on collaborative+taxonomic evidence alone.**

Deployment consequence. Because the POG/Spotify content signals are *functions of these encoders' outputs*, serving the method on a genuinely new item requires first running the same encoders (BLIP on the item's image and description and BGE on its title for fashion, or CLAP on the track for music) to embed it, before any content/soft-category signal can place it. The TF–IDF datasets need no external model — only the item's text. We state this so the true dependency footprint is explicit: the "zero-parameter" claim is about the *scorer* (it fits no weights by gradient descent), not about these upstream feature extractors, which are a real deployment prerequisite wherever deep content features are used.

2.3 The side labels: categories, artists, albums

Each item also has discrete **metadata labels**. For fashion that is an explicit category id `cate` (e.g. "boots"); for music it is the `artist_uri` and `album_uri`. We store them as integer label vectors $\text{cat} \in \{0, \ldots, C-1\}^n$ (and, for music, alb). When a dataset has *no* usable category, the code manufactures one by running k-means (400 clusters) on the multimodal embedding and using the cluster id as a pseudo-category (`eval/fullrank.py`). These labels feed the *taxonomic* signals (§5.2).

3 Three structures we derive immediately

Everything collaborative is built from three quantities read straight off B.

(1) Item frequency f. How many training bundles contain each item:

$$f = B^\top \mathbf{1}_m \in \mathbb{Z}^n, \qquad f_i = \#\{\text{bundles containing } i\}.$$

An item with $f_i = 0$ is **cold** (never seen in training); cold items are exactly where the collaborative signals go silent and the prior has to take over (§7). In code: `self.freq = B.sum(0)`.

(2) Co-occurrence matrix C. How often each *pair* of items shares a bundle, with the diagonal removed (we never "co-occur" an item with itself):

$$C = B^\top B, \qquad C_{ii} \leftarrow 0.$$

C_{ci} = number of training bundles containing both c and i. In code: `self.C = (B.T @ B)` then `C.setdiag(0)`.

(3) Bundle sizes. $|b_j| = \sum_i B_{ji}$, used to down-weight large bundles (a pair that co-occurs inside a 20-item playlist is weaker evidence than a pair that co-occurs inside a 2-item set). In code: `self.bsize`.

Example 2 (the three structures: sketch shape, then the real query). On the 4-item sketch, from B in Example 1:

$$f = \begin{pmatrix}2\\3\\2\\1\end{pmatrix}, \qquad B^\top B = \begin{pmatrix}2&2&1&1\\2&3&2&1\\1&2&2&0\\1&1&0&1\end{pmatrix} \xrightarrow{\text{zero diag}} C = \begin{pmatrix}0&2&1&1\\2&0&2&1\\1&2&0&0\\1&1&0&0\end{pmatrix}.$$

Read C as: items 1 and 2 share 2 bundles; items 3 and 4 never share one.
On the real query, these same structures hold the actual numbers we will use throughout. The seeds' co-occurrence degrees (row sums of the real C) are $d_{\text{bag}} = 46$ and $d_{\text{sandals}} = 11$, the grand total is $T = \sum_{ij} C_{ij} = 193{,}688$, and the two held-out items sit at opposite extremes of C: the **earrings** co-occur with the bag in $C[\text{earrings}, \text{bag}] = 15$ training outfits and with the sandals in 2, whereas the **dress** co-occurs with *neither* ($C[\text{dress}, \cdot] = 0$), having appeared in only $f_{\text{dress}} = 2$ outfits total. Those few integers already decide most of what follows.

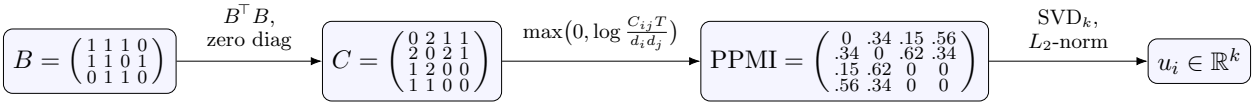


Figure 3. *The collaborative pipeline, built once from the bundles alone* (4-item sketch to show the shapes; the real $B, C,$ PPMI are 31,217-square). The bundle–item matrix B becomes the item–item co-occurrence $C = B^\top B$ (diagonal zeroed); C becomes the chance-corrected PPMI (§4.2); and a truncated SVD of PPMI gives each item a dense "collaborative fingerprint" u_i (§4.3). The partial bundle is *not* involved yet: these are fixed structures a query is later scored *against*.

4 The vocabulary of operations (general idea → local definition → example)

Five operations recur throughout. Each is a standard idea, but what matters is *the exact quantity it is applied to here*. This section pins that down; §5 then just assembles them.

4.1 Co-occurrence scoring: "who shows up with the seeds"

General. The simplest bundle signal: score a candidate c by how often it co-occurred with the seed items. If $x_S \in \{0,1\}^n$ is the **seed indicator** ($(x_S)_i = 1$ iff $i \in S$), then

$$\mathrm{cooc}(c \mid S) = \sum_{i \in S} C_{ci} = (C\,x_S)_c.$$

A matrix–vector product: the score vector is column-combination of C selected by the seeds.

Here. Exactly this, with C the zero-diagonal co-occurrence matrix above. In code the seed indicators for a whole batch of queries are stacked into a sparse matrix `Ind` ($n \times$ nb, one column per query) and the batch of score vectors is the single product `C @ Ind` (in `score_all_signals`, `eval/fullrank.py`). This "$W\,x_S$" shape is the template for *every* collaborative signal.

Example 3 (co-occurrence on the real query). For our query $S = \{\text{bag}, \text{sandals}\}$, x_S is the 31,217-long indicator with a 1 in the bag and sandals slots, and $\mathrm{cooc}(c \mid S) = (Cx_S)_c = C[c, \text{bag}] + C[c, \text{sandals}]$ simply sums the two seed columns. Using the real counts from Example 2,

$$\mathrm{cooc}(\text{earrings} \mid S) = 15 + 2 = \mathbf{17}, \qquad \mathrm{cooc}(\text{dress} \mid S) = 0 + 0 = \mathbf{0}.$$

So raw co-occurrence *already* nominates the held-out earrings strongly and is completely *blind* to the held-out dress (which never shared an outfit with either seed). Hold this: co-occurrence rewards whatever ships together, including popular hubs that ship with everything—PMI will next discount exactly that.

$$\underbrace{\begin{pmatrix} 0 & 2 & 1 & 1 \\ 2 & 0 & 2 & 1 \\ 1 & 2 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}}_{C} \quad \underbrace{\begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}}_{x_S,\ S=\{1\}} \quad = \quad \underbrace{\begin{pmatrix} \cdot \\ 2 \\ 1 \\ 1 \end{pmatrix}}_{\mathrm{score}(\cdot \mid S)}$$

Figure 4. *Filling a partial bundle is one matrix–vector product.* The seed indicator x_S carries a 1 in each seed slot, so Cx_S **sums the seed columns of** C. For $S = \{1\}$ it reads off column 1: candidate 2 scores 2, candidates 3, 4 score 1; the seed itself ($c = 1$, shown "·") is masked to $-\infty$ before ranking. *Every* collaborative signal has this exact shape Wx_S—only the matrix W changes (raw C, PPMI, the degree-normalised W_{pair}, or the household operator). (Matrix shown is the 4-item sketch; on the real query the same product gives cooc(earrings) = 15+2 = 17, Ex. 3.)

4.2 PMI / PPMI: "is this pair together *more than chance*?"

General. Raw co-occurrence is biased toward popular items: a hub co-occurs with everything. Pointwise Mutual Information corrects for that by comparing the *joint* probability of a pair to the product of their *marginals*:

$$\mathrm{PMI}(i,j) = \log \frac{P(i,j)}{P(i)\,P(j)}.$$

It is positive when i, j appear together more than independence predicts, negative when less. The *positive* part, $\mathrm{PPMI} = \max(0, \mathrm{PMI})$, keeps only genuine associations (and is what makes the matrix sparse and well-behaved for the SVD below).

Here — and this is the subtle part. The probabilities are estimated **from the co-occurrence matrix C itself**, not from item frequencies f. Let

$$d_i = \sum_j C_{ij} \quad \text{(row sum of } C\text{, the "co-occurrence degree" of } i\text{)}, \qquad T = \sum_{i,j} C_{ij} \quad \text{(grand total)}.$$

Then the *joint* is $P(i,j) = C_{ij}/T$ and the *marginals* are $P(i) = d_i/T$, giving

$$\mathrm{PPMI}_{ij} = \max\Big(0,\ \log \frac{C_{ij}\,T}{d_i\,d_j}\Big), \qquad d_i = \textstyle\sum_j C_{ij}, \quad T = \textstyle\sum_{i,j} C_{ij}.$$

Note what the "marginal" is here: the co-occurrence degree d_i (how much total pairing mass item i carries), not the bundle-frequency f_i. These differ—e.g. a popular item in many *singleton-ish* bundles has high f but modest d. The code (`operators.py`:103–107) uses precisely d and T derived from C: `freq_c = C.sum(1)`, `totc = C.sum()`. The same construction is reused verbatim for *category* co-occurrence and for *soft-cluster* co-occurrence (§5.2)—same formula, different "vocabulary" (items, categories, or clusters).

Example 4 (PPMI on the real query, and why popularity must be divided out). Using the real degrees and total from Example 2 ($d_{\text{earrings}} = 1635$, $d_{\text{bag}} = 46$, $d_{\text{sandals}} = 11$, $T = 193{,}688$):

$$\mathrm{PPMI}(\text{earrings}, \text{bag}) = \max\Big(0,\ \log \tfrac{C_{ij}\,T}{d_i\,d_j}\Big) = \max\Big(0,\ \log \tfrac{15\cdot 193688}{1635\cdot 46}\Big) = \max(0, \log 38.6) = \mathbf{3.65},$$

and $\mathrm{PPMI}(\text{earrings}, \text{sandals}) = \log \frac{2\cdot 193688}{1635\cdot 11} = \log 21.5 = 3.07$, so $\mathrm{ppmi}(\text{earrings} \mid S) = 3.65 + 3.07 = \mathbf{6.72}$. The dress, having no co-occurrence, stays at 0.

The popularity correction, in one number. A *popular* pair of pearl earrings (a hub with degree $d = 9{,}198$) also shares exactly 2 outfits with the sandals—the *same raw count* as the true earrings. Yet PMI credits it far less,

$$\mathrm{PPMI}(\text{pearl}, \text{sandals}) = \log \tfrac{2\cdot 193688}{9198\cdot 11} = \log 3.83 = \mathbf{1.34} \qquad \text{(vs. 3.07 for the true earrings)},$$

because a hub that co-occurs with *everything* was already expected to co-occur with the sandals by chance. That 3.07-vs-1.34 gap, from identical raw counts, is the entire point of PMI—and why we keep a PPMI channel alongside raw co-occurrence. (The seed *bag*'s own degree $d_{\text{bag}} = 46$ is modest, so it is *not* a hub; the hub here is the popular pearl-earring listing.)

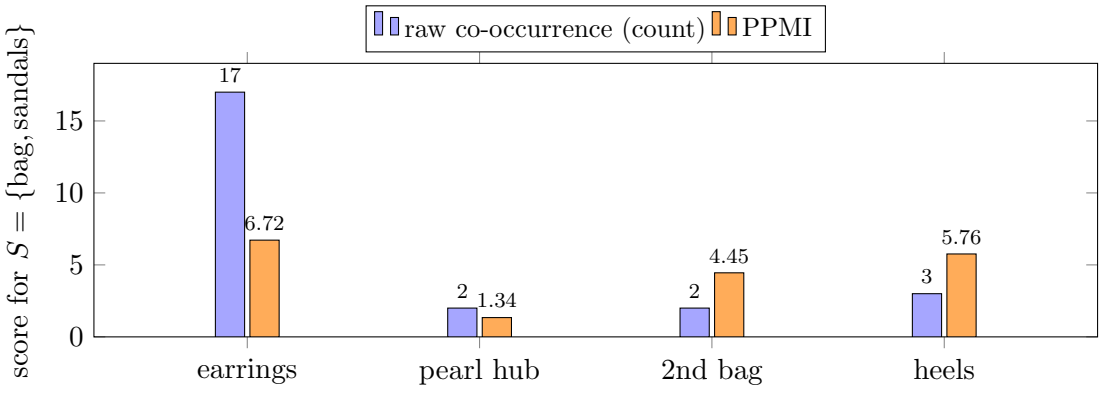


Figure 5. *Real query: popularity is divided out.* Raw co-occurrence counts (blue) and PPMI (orange) for four candidates. The true **earrings** dominate both. The crucial comparison is the **pearl hub**: it shares the *same* raw count with the sandals as the true earrings do (2 outfits), but because it is a popularity hub (degree 9,198) PPMI credits that share only 1.34 versus the earrings' 3.07 — the held-out dress (not shown) scores 0 on both, being seen in only 2 outfits total. Standardisation (§6) then puts these on a common scale; what survives into the blend is the *ordering*.

4.3 Truncated SVD: a dense "collaborative fingerprint" per item

General. A truncated singular value decomposition approximates a matrix by the best rank-k product $M \approx U\Sigma V^\top$; the rows of $U\Sigma$ (here we keep U) are a k-dimensional *embedding* of the objects, capturing their dominant patterns of variation while discarding noise.

Here. The SVD is taken of the **item–item PPMI matrix**, *not* of a user–item rating matrix. With $k = 128$ (`TruncatedSVD(n_components=128)` on `self.PPMI`, `operators.py`:108–111), we get a 128-dimensional vector per item and L_2-normalise each row:

$$U = \mathrm{SVD}_{128}(\mathrm{PPMI}), \qquad u_i = U_i / \|U_i\|_2.$$

Think of u_i as item i's "collaborative fingerprint": two items get similar fingerprints when they pair with *similar sets of partners*, even if they never co-occur *directly*. This is the only embedding the method builds itself, and it is built from co-occurrence alone—no external features. It is stored as `self.Ucsvd` and exposed as the signal `csvd`.

Remark 1 (direct vs. indirect association). Raw co-occurrence and PPMI see only *direct* pairs ($C_{ci} > 0$). The SVD fingerprint sees *second-order* structure: items 3 and 4 never co-occur ($C_{34} = 0$), yet both co-occur with $\{1, 2\}$, so their fingerprints are pulled closer. This is how the collaborative side reasons about pairs it has never literally observed together.

On the real query. The seed centroid is $\frac{1}{2}(u_{\text{bag}} + u_{\text{sandals}})$; scoring the held-out items against it gives csvd(earrings) = 0.38 and csvd(dress) = **0.62**. Notice the fingerprint actually rates the *rare dress higher*: even with only $f = 2$ outfits, the dress keeps similar *company* to the seeds in PPMI-space, which is exactly the indirect evidence a direct count misses. (As it happens, validation prunes `csvd` to weight 0 on POG—`simp` and `cat` already cover it—so this particular signal does not enter the final blend; but it shows the fingerprint seeing what raw co-occurrence cannot.)

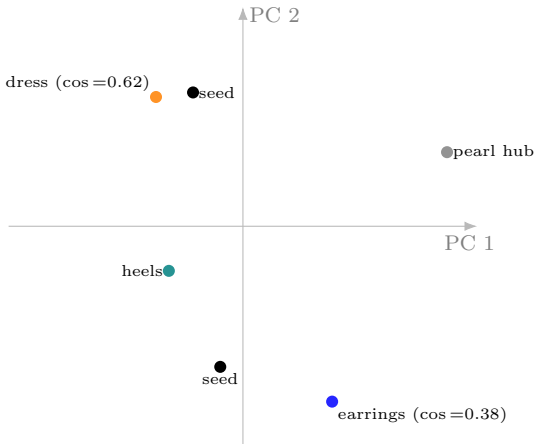


Figure 6. *Real collaborative fingerprints, projected to 2D.* The 128-dim PPMI-SVD fingerprints u_i of the seeds and candidates, reduced to their first two principal components. The held-out **dress** sits *closer* to the seed centroid (cos =0.62) than the **earrings** (cos =0.38): even with only 2 training outfits, the dress keeps similar *company* to the seeds in co-occurrence space — the partners-of-partners evidence a direct count misses. (`csvd` is nonetheless pruned to weight 0 on POG, since `simp` and `cat` already carry the query.)

4.4 Degree normalisation: taming hubs and big bundles

General. Popular items and large bundles dominate raw sums. We damp them with two monotone discounts.

Here. Two fixed vectors (`operators.py`:79–81), with smoothing constants read directly from code:

$$\underbrace{(f_i + 5)^{-0.5}}_{\text{item discount}} \qquad \text{and} \qquad \underbrace{(|b| + 1)^{-0.5}}_{\text{bundle-size discount}} \quad .$$

The "+5" and "+1" are smoothing (they stop rare items/small bundles from blowing up); the exponent 0.5 is a square-root taming. These assemble into the **degree-normalised pairwise operator**

$$W_{\text{pair}} \;=\; B^{\top}\,\mathrm{diag}\big((|b|+1)^{-0.5}\big)\,B\,\mathrm{diag}\big((f+5)^{-0.5}\big), \qquad \mathrm{diag}(W_{\text{pair}}) = 0.$$

Reading it right-to-left: scale each seed column by its item discount, walk bundle→items weighting each bundle by its size discount, land back on items. So $W_{\text{pair}}(c \mid S) = \sum_{b \ni c}(|b|+1)^{-0.5} \sum_{s \in S \cap b}(f_s + 5)^{-0.5}$: a *smoothed, size-aware* co-occurrence. Setting both exponents to 0 recovers plain C (the case used in Examples 3–4 for clarity).

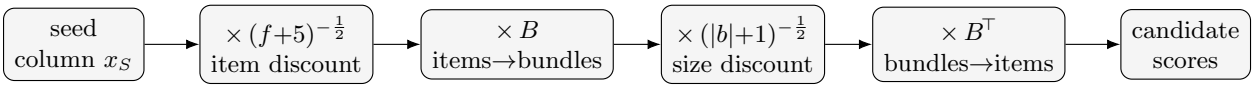


Figure 7. $W_{pair} = B^{\top}\mathrm{diag}((|b|{+}1)^{-1/2})\,B\,\mathrm{diag}((f{+}5)^{-1/2})$*, read right-to-left as a walk.* A seed's mass is first shrunk by its own popularity, carried up to the bundles it sits in (each discounted by its size), then back down to candidates. On the 4-item sketch the item discounts are $(f{+}5)^{-1/2} = (0.378, 0.354, 0.378, \mathbf{0.408})$: the *rarest* item 4 is damped least and the *hub* item 2 most—the same chance-correction PPMI applies, here multiplicative. Both exponents set to 0 recovers plain C.

4.5 Graph propagation: a two-hop smoothing walk

General. "Spread" a score across the item graph so that an item inherits a little evidence from its graph neighbours—one round of label propagation.

Here. The bipartite item↔bundle graph gives a zero-diagonal, symmetrically normalised one-step operator applied as a *factored* product (never materialised as a dense $n \times n$ matrix, which matters at n = 254,000): `propagate(X)` computes $B \to$ items then back, with the item discount on both ends and the self-term subtracted (`operators.py`:121–131). It is used two ways: (i) to produce a *two-hop* version `diff2` of the pairwise score, and (ii) to build the content "bridge" (§5.3) by smoothing content vectors over the co-purchase graph.

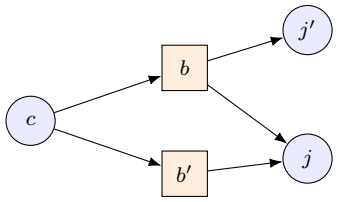


Figure 8. *One propagation step spreads a score two hops across the co-purchase graph.* Item c's evidence flows out to the bundles it belongs to and back down to *their* other members j, j' (both ends carrying the item discount, the self-term subtracted). Applied to the pairwise score it yields the two-hop signal `diff2`; applied to content vectors it builds the `bridge` (§5.3). It is the factored product $B^\top(\cdots)B$, never a dense $n \times n$ matrix—essential at $n = 254{,}000$.

5 The signals, built one by one

A **signal** is one scoring function $z_k(c \mid S)$: one opinion about each candidate. BundleFuse computes about eleven of them, in three families. For each we give *what it captures*, *the exact construction*, and *which embedding (if any) it uses*. (Code: `score_all_signals`, `score_all_signals` in `eval/fullrank.py`.)

5.1 Collaborative family (from B only — no embeddings)

`cooc` Raw co-occurrence $C\,x_S$ (§4, Ex. 3). "How often did it ship with the seeds."

`ppmi` Chance-corrected co-occurrence PPMI x_S (§4.2, Ex. 4). "... more than chance would predict."

`csvd` Similarity to the seeds' average collaborative fingerprint: with U the PPMI-SVD embedding (§4.3), score candidate c by $\langle u_c,\ \frac{1}{|S|}\sum_{s\in S} u_s\rangle$. Captures *indirect* collaboration.

`simp` The degree-normalised pairwise score $W_{\text{pair}}x_S$ (§4.4) *plus* a higher-order "incidence surplus" Δ_{inc} described next.

`diff2` The two-hop propagated version of `simp` (§4.5): neighbours-of-neighbours.

`pop` Pure popularity, $\log(1 + f_c)$, identical for every query. A backstop that knows nothing about S.

The higher-order incidence surplus Δ_{inc}. This is the one genuinely "set-level" collaborative term. Ordinary co-occurrence is *pairwise*: it treats S as a bag of seeds and sums seed-by-seed. Δ_{inc} instead rewards a candidate for appearing in training bundles that already contain *two or more* of the current seeds together (`score_simplicial_incidence_only`, `operators.py`:149–164):

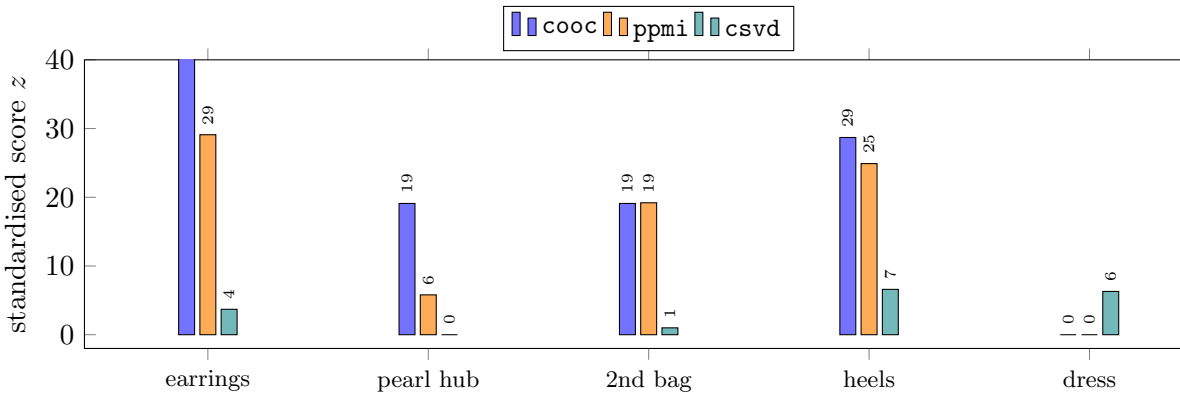
$$\Delta_{\text{inc}}(c \mid S) = \sum_{b \ni c} (|b| + 1)^{-0.5} \Big(\sum_{s \in b \cap S} (f_s + 5)^{-0.5} \Big) \cdot \beta \cdot \max\big(0,\ |b \cap S| - 1\big),$$

with $\beta = 1.5$. The factor $\max(0, |b \cap S| - 1)$ is the crux: it is **zero unless a bundle overlaps the seeds in at least two items**, so $\Delta_{\text{inc}} \equiv 0$ for single-seed queries ($|S| = 1$) and whenever no training bundle shares ≥ 2 seeds with S. It is a surplus *on top of* the pairwise term, not a replacement. *Honesty note: in our ablations this term moves Recall@20 by at most +0.79 points once the prior is in place; it is near-inert, and we report that rather than overselling it.*

Second-order "concordance" R_γ (Rao). A second set-level term (signal `rao`) scores a candidate by how strongly it is *simultaneously* pulled by *distinct, non-redundant* pairs of seeds, using Rao's quadratic-entropy kernel to discount seeds that are near-duplicates of each other (same category / similar embedding). With per-seed pulls $W_{\text{pair}}[\,\cdot, s]$ and a seed–seed redundancy kernel K_S,

$$R_\gamma(c \mid S) = \frac{\sum_{s \neq s'} K_S(s, s')\, W_{\text{pair}}[c, s]\, W_{\text{pair}}[c, s']}{\left(\sum_s W_{\text{pair}}[c, s]\right) + \tau},$$

(`score_rao_concordance`, `operators.py`:179–217; $\tau = 0.05$). The kernel K_S is built from the seeds' category labels *and* their content embeddings, so this signal *does* touch embeddings. Like Δ_{inc} it is a small corrective, reported honestly as such.



(`cooc` caps at 162 for the earrings — off the top of this axis — so the bars are clipped at 40 to keep the others visible)

Figure 9. *Three collaborative opinions on the real query, and they disagree.* Standardised `cooc`, `ppmi` and `csvd` (z-scores over all 31,217 items). Raw co-occurrence and PPMI back the **earrings** (they literally ship together). But the SVD fingerprint `csvd` backs the **heels** and even the held-out **dress** (both ≈ 6) over the earrings (3.8): it sees *partners-of-partners*, so it rates the rare dress highly despite its zero co-occurrence. No single collaborative signal is right everywhere — which is exactly why fusion (§7) is needed.

5.2 Taxonomic family (from category/artist/album labels)

These answer "what *kinds* of things go with these kinds of things," and crucially they work for **cold** items (an unseen pair of jeans still has the category "jeans").

`cat` **Category complementarity**, via PMI *between categories*. Build a bundle×category count matrix T_c (how many items of each category each bundle holds), form the category co-occurrence $T_c^\top T_c$ (diagonal zeroed), and apply the *same PPMI formula* as §4.2 but over categories, giving $P_{\text{cat}} \in \mathbb{R}^{C \times C}$ (`operators.py`:262–269). Score a candidate by summing, over the seeds' categories, the category-PMI into the candidate's category: $\text{cat}(c \mid S) = \sum_{s \in S} P_{\text{cat}}[\text{cat}(s), \text{cat}(c)]$. ("Shirts complement trousers" emerges as a large P_{cat} entry.)

slot self-exclusion A *signed* correction added into `cat`: for a category that almost never repeats inside a bundle (you rarely put two hats in one outfit), it *subtracts* score from other items of that same category, discouraging duplicate slots (`score_slot_saturation`, `operators.py`:307–316). The threshold uses an empirical per-category duplication rate.

`homo` (music) **Homophily**: reward candidates by the same *artist* or *album* as the seeds, with album weighted twice as much as artist (`eval/fullrank.py`). Normalised by $\sqrt{\text{degree}}$ so prolific artists do not swamp the score.

`softcat` (fashion) **Soft/fuzzy categories**: run k-means (400 clusters) on the multimodal embedding, give each item a *soft* assignment (a softmax of cosine-to-centroids), and apply the same PPMI construction over the 400 clusters. This captures sub-type complementarity finer than the official category labels. **Uses embeddings** (for the clustering and soft assignment).

On the real query. Our seeds are categories "bags" and "sandals"; the earrings are "earrings." The category-PMI votes add over the seeds' categories:

$$\text{cat}(\text{earrings} \mid S) = P_{\text{cat}}[\text{bags}, \text{earrings}] + P_{\text{cat}}[\text{sandals}, \text{earrings}] = 1.33 + 1.97 = \mathbf{3.30},$$

and the dress scores $1.25 + 1.93 = \mathbf{3.18}$. **Both held-out items are strong category complements**—and crucially taxonomy rates the rare dress *just as highly* as the common earrings, with *no co-occurrence of either*. That is the whole promise of this family. Slot self-exclusion then earns its keep on a tempting distractor: a *second bag* (same category as a seed) would get $P_{\text{cat}}[\text{sandals}, \text{bags}] = 1.18$, but because an outfit almost never takes two bags the duplicate-slot term subtracts 0.5, to 0.68—below the field average, so its standardised `cat` score is *negative* ($z = -1.01$). Taxonomy actively vetoes the second bag. (Why, then, is the dress not recovered? Because `cat` is only one weighted vote; §7 shows the earrings *also* carry a huge collaborative `simp` score that the dress lacks.)

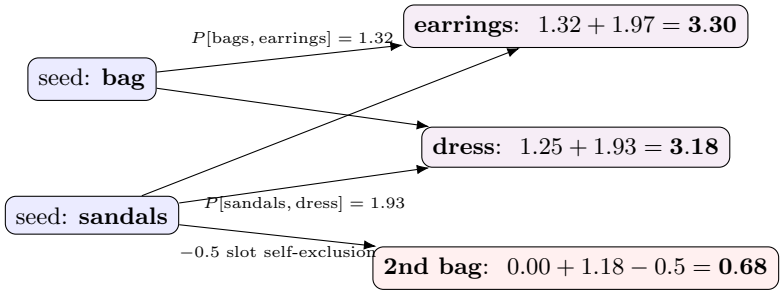


Figure 10. *Taxonomic scoring on the real query: category-to-category PMI.* The score of a candidate sums, over the seeds' categories, the category-PMI into the candidate's category. Both held-out items are strong category complements — **earrings** 1.32+1.97=3.30 and **dress** 1.25+1.93=3.18 — and crucially taxonomy rates the *rare* dress as highly as the common earrings, with *no co-occurrence of either*. A tempting **second bag** (same category as a seed) would score 1.18, but the slot self-exclusion term subtracts 0.5 (outfits rarely take two bags), leaving 0.68 — below the field, so its standardised `cat` is negative. Taxonomy vetoes it.

5.3 Content family (from the embedding vectors)

This family is where the §2.2 embeddings do their work. All three signals score a candidate by the cosine similarity of its (processed) content vector to the *average* content vector of the seeds: $\text{score}(c \mid S) = \langle \phi_c,\ \frac{1}{|S|} \sum_{s \in S} \phi_s \rangle$ for some processed feature ϕ (`eval/fullrank.py`). They differ in *which* ϕ:

`style` / `imgstyle` / `audiostyle` **Category-disentangled** content. Raw embeddings are dominated by "what category is this" (all boots look like boots), which is uninformative for

within-outfit matching. `disentangle` (`operators.py`:323–330) subtracts the per-category mean embedding and renormalises, leaving only the *style* residual ("this boot's particular look"). Built separately for the title embedding (`style`), the image embedding (`imgstyle`), and the audio embedding (`audiostyle`).

`bridge` **Graph-bridged** content. Take the disentangled style vectors and smooth them one hop over the co-purchase graph (§4.5), then add that back: $\phi_{\text{bridge}} = \text{normalise}(\text{style} + 0.5 \cdot \mathcal{P}\,\text{style})$ (`build_dense_bridge`, `operators.py`:332–337). This lets content similarity borrow a little collaborative structure—items that are both *look-alike* and *bought-with-alike* score highest.

(optional) `contrast` A tiny (≤ 0.30M-parameter) set-attention encoder trained with a contrastive loss. **It is off by default** (`ENCODER=0`); the whole point of the paper is that the zero-parameter signals above are enough. When off, it contributes nothing.

On the real query. Content is the weakest family *here*, and honestly so. After category-disentangling, the earrings' `style` cosine to the seed centroid is ≈ 0.00 and the dress's is 0.06—the *look* of a bag-and-sandals pairing barely predicts either answer's texture. The graph-bridged `bridge` is more useful: it scores the earrings 0.23 (they are co-purchased, so the one-hop smoothing lifts them) but the dress -0.03. So on this outfit the content family mostly abstains; it earns its weight on the datasets and queries where *look-alike* is the signal (e.g. clothing), which is why validation still keeps a small $w_{\texttt{bridge}} = 0.25$.

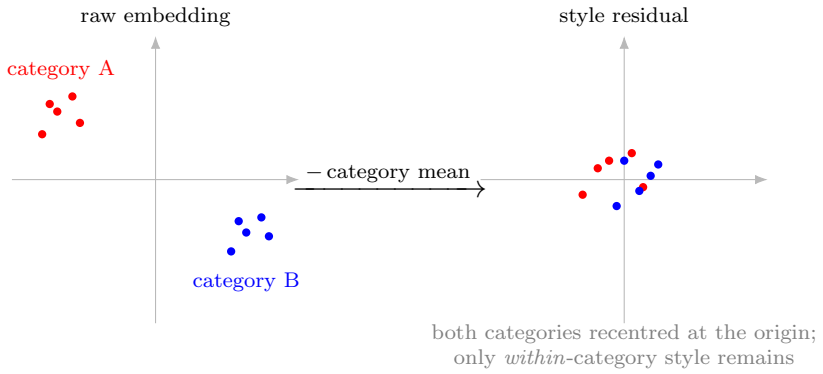


Figure 11. *What* `disentangle` *does.* Left: raw embeddings separate mostly by *category* (all of A over here, all of B over there)—useless for matching *within* an outfit, since the seeds and the answer are different categories by design. Subtracting each category's mean vector (right) cancels that bulk separation and leaves only each item's *style* deviation, which is what the content signals actually compare.

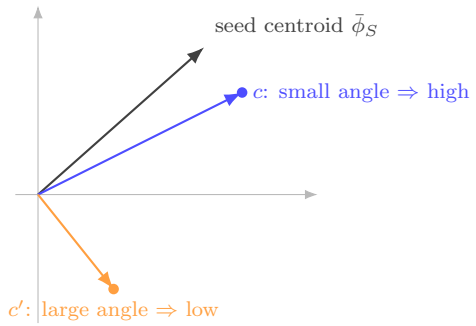


Figure 12. *Content scoring is cosine to the seeds' average style vector* (schematic—the 4-item sketch carries no embeddings; the real query's content values are `style` ≈ 0, `bridge` $= 0.23$ for the earrings). Each content signal (`style`/`imgstyle`/`bridge`) averages the seeds' processed vectors into a centroid $\bar{\phi}_S$ and ranks candidates by $\langle \phi_c, \bar{\phi}_S \rangle$. Because the vectors are category-disentangled (Fig. 11), this rewards *within-outfit style match*; like taxonomy it uses no co-occurrence, so it too scores **cold** items.

6 From a signal to per-query numbers: scoring and standardising

Every signal above ultimately produces, for a batch of queries, a matrix $R_k \in \mathbb{R}^{n \times \mathrm{nb}}$: one score per (item, query). Collaborative signals get there by the "W Ind" product (§4); content and `csvd` signals by the seed-centroid dot product (§5.3).

Why we must standardise. These raw scores live on wildly different scales—co-occurrence counts are integers in [0, tens], cosine similarities are in $[-1, 1]$, PMI values are small positive logs. Adding them directly would let whichever signal happens to have the biggest numbers dominate. So each signal is **column-standardised** (*per query*, across the n candidates) to mean 0, standard deviation 1:

$$z_k(c \mid S) = \frac{R_k(c \mid S) - \mathrm{mean}_{c'} R_k(c' \mid S)}{\mathrm{std}_{c'} R_k(c' \mid S) + 10^{-9}}.$$

(Code: `zcols`, `operators.py`:32–36.) After this, every signal speaks the same language—"how many standard deviations above the field is this candidate, according to me"—and the weights in the next section are comparing like with like.

On the real query (why the magnitudes look so large). The collaborative signals are extremely *peaked*: of the 31,217 items almost all score ≈ 0 on `simp`, so its standard deviation is tiny (0.0144). The earrings' raw `simp`$= 1.79$ therefore standardises to $z = (1.79 - 0.0003)/0.0144 \approx$ **124**—124 standard deviations above the field. Taxonomy is far flatter (mean 1.61, std 0.92), so the earrings' raw `cat`$= 3.30$ becomes a modest $z = (3.30 - 1.61)/0.92 \approx 1.83$. The huge `simp` number is not a bug: it says "*nothing else in the catalogue co-occurs with these seeds like the earrings do*," and standardisation is exactly what lets that peaked collaborative verdict and the gentle taxonomic one be *added* on a common scale in the blend below.

7 Fusion: four stages to the final score

Before the formulas, here is the whole machine on one page: the fixed inputs (left) feed the three signal families, which are standardised and then combined into *two* aggregates—a collaborative score Y and a cold-start prior μ_0—that the final routing step mixes per candidate according to how often that candidate was seen.

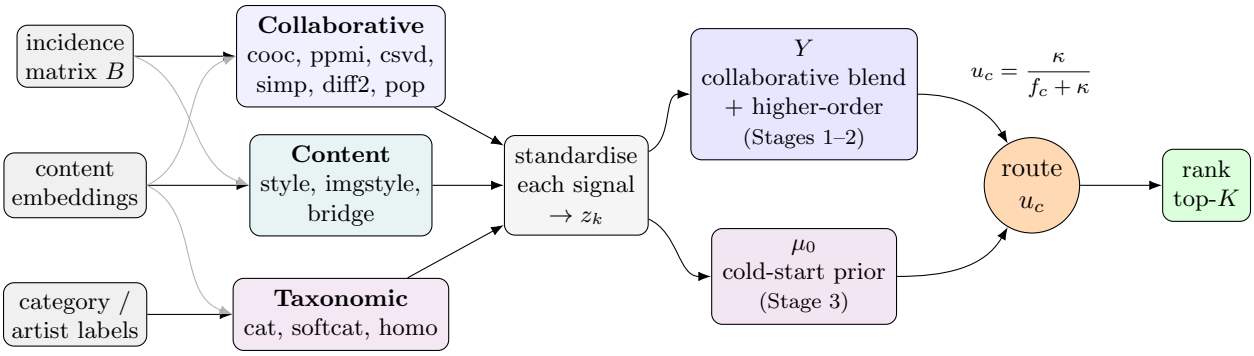


Figure 13. *Solid arrows are each family's main input; gray arrows are secondary ones* (e.g. `bridge` also reads B; `softcat` also reads embeddings). f_c is the candidate's frequency, so cold candidates ($f_c = 0$) route entirely to μ_0. We now unpack the four stages.

Now we combine the standardised signals $\{z_k\}$ into one number per candidate. There are four stages; the only quantities "learned" are a handful of scalars, each chosen by **greedy search on a validation split** to maximise NDCG@20 (`fit_simba` in `eval/fullrank.py`). §7.1 shows the result is insensitive to *which* cutoff we validate at, and why a cutoff-free objective would be worse.

Stage 1 — weighted blend. Pick a weight $w_k \geq 0$ per signal and form

$$Y_{\text{base}}(c \mid S) = \sum_k w_k \, z_k(c \mid S).$$

The weights are found by *greedy coordinate ascent*: sweep each w_k over the grid $\{0, 0.25, 0.5, 1, 1.5\}$, keep the change if validation NDCG@20 improves, repeat up to four passes (the greedy loop in `fit_simba`). A signal that never helps simply gets weight 0—the method prunes itself. Afterwards a backward-elimination pass tries to *drop* optional signals (bridge, contrast, softcat, homo) if doing so does not hurt validation (`prune_weights_val`).

Stage 2 — fold in the set-level terms. Blend the pairwise base with its incidence-augmented version and the Rao term, via two more validation-picked scalars $\alpha \in \{0, 0.5, 1\}$ and $\gamma \in \{0, 0.2, 0.35\}$:

$$Y(c \mid S) = (1 - \alpha)\, Y^{\text{pair}}_{\text{base}} + \alpha\, Y^{\text{inc}}_{\text{base}} + \gamma\, z_{\text{rao}}.$$

Here $Y^{\text{pair}}_{\text{base}}$ uses the *purely pairwise* versions of `simp`/`diff2` and $Y^{\text{inc}}_{\text{base}}$ uses the *incidence-augmented* ones, so α literally dials how much of the higher-order surplus to admit. (On most datasets α, γ come out small—the honest "near-inert" finding.)

Stage 3 — the cold-start prior μ_0. Build a *second* score that needs no collaborative exposure of the candidate, from the metadata/content signals only:

$$\mu_0(c \mid S) = (\text{frozen scale}) \times \Big(\sum_{k \in \text{prior keys}} w_k \, z_k + \gamma \, z_{\text{rao}} \; [+ \, w_{\text{pop}} z_{\text{pop}}] \Big),$$

where "prior keys" are the taxonomic/content signals that earned positive weight (`cat`, `softcat`, `style`, `imgstyle`, `bridge` for fashion; `cat`, `homo`, `audiostyle`, `bridge` for music). The "frozen scale" is a single number = std(Y)/std(raw prior) computed **on the validation set and then frozen**, so that at test time μ_0 is on the same scale as Y and— critically—*a candidate's score never depends on which other queries happen to be in its test batch* (the `frozen_prior_scale` computed in `fit_simba`). This is why the method is exactly reproducible.

Stage 4 — frequency-confidence routing (shrinkage). Combine the collaborative blend Y and the prior μ_0 per candidate, trusting Y more when the candidate is *frequent* and μ_0 more when it is *rare*:

$$\hat{\theta}(c \mid S) = (1 - u_c)\, Y(c \mid S) + u_c\, \mu_0(c \mid S), \qquad u_c = \frac{\kappa}{f_c + \kappa}.$$

The single dial $\kappa \geq 0$ is picked on validation (`conjugate_posterior`, `operators.py`:526–543). Read the two extremes:

- **Cold candidate**, $f_c = 0 \Rightarrow u_c = 1 \Rightarrow \hat{\theta} = \mu_0$: ranked purely by taxonomy/content, since it has no co-occurrence evidence at all.
- **Very frequent candidate**, $f_c \gg \kappa \Rightarrow u_c \to 0 \Rightarrow \hat{\theta} \to Y$: ranked by the collaborative evidence, which for a hub is plentiful and trustworthy.

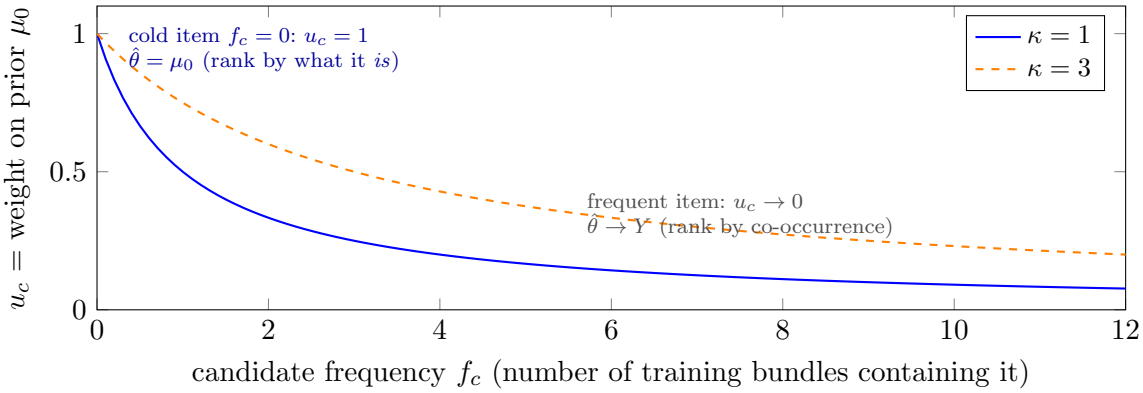


Figure 14. *The routing dial.* As a candidate is seen more often, weight slides off the prior and onto the collaborative evidence. The single learned scalar κ sets *how fast*: larger κ (orange) keeps trusting the prior up to higher frequencies. It is chosen on validation, not derived from a model. The *form* of this rule is the posterior mean of a Normal–Normal model (derived in `formalization.tex`); but we do *not* estimate κ from that model—it is just selected on validation—so in practice Stage 4 is a principled, frequency-aware interpolation between "what co-occurrence says" and "what the item *is*." Finally we rank $\mathcal{I} \setminus S$ by $\hat{\theta}(\cdot \mid S)$ and return the top K.

Example 5 (the four stages on the real query). Validation selected $w_{\text{simp}} = w_{\text{cat}} = 1$ and $w_{\text{softcat}} = w_{\text{bridge}} = w_{\text{pop}} = 0.25$ (all other weights 0), $\alpha = 1$, $\gamma = 0$, $\kappa = 0$. **Stages 1–2 (blend Y).** Standardising each signal and summing with these weights, the earrings get

$$Y = \underbrace{1{\cdot}124.1}_{\texttt{simp}} + \underbrace{1{\cdot}1.83}_{\texttt{cat}} + 0.25\big(\underbrace{3.94}_{\texttt{softcat}} + \underbrace{3.42}_{\texttt{bridge}} + \underbrace{9.8}_{\texttt{pop}}\big) = \mathbf{130.2},$$

while for the dress every collaborative term is 0, leaving $Y = 0 + 1.71 + 0.25(-0.74 - 0.58 + 0.6) = \mathbf{1.52}$—essentially its lone `cat` vote. **Stage 3 (prior μ_0).** The taxonomy/content prior likes both category complements: $\mu_0 = 8.2$ (earrings), 2.1 (dress). **Stage 4 (route).** Validation chose $\kappa = 0$ here, so $u_c = \kappa/(f_c + \kappa) = 0$ and $\hat{\theta} = Y$ for every candidate—*no shrinkage on this dense dataset.* Final full-catalogue ranks: **earrings** 1, dress 3971. The earrings win not because taxonomy prefers them (it rates both ≈ 3.2) but because they *add* a huge collaborative `simp` the dress cannot; and the one mechanism that could have lifted the dress—shrinkage toward the prior that liked it ($\mu_0 > Y$)—is switched off on dense POG. Every number is a fixed function of S: no randomness, no test-batch dependence.

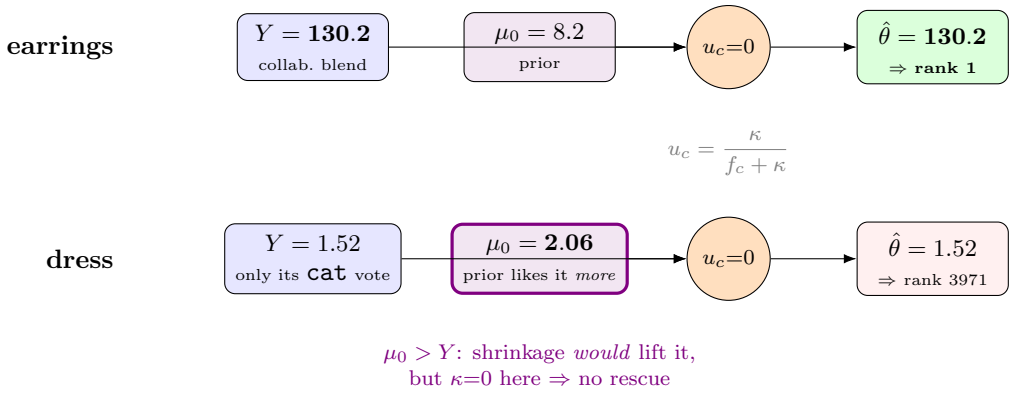


Figure 15. *The routing stage on the real query, for the two held-out items.* The earrings' collaborative blend Y=130.2 dwarfs everything, so they rank 1. The dress's blend is only Y=1.52—it has no co-occurrence, so only its category vote survives (§5.2). Its prior μ_0=2.06 actually *likes it more* than Y, which is exactly the evidence frequency-confidence shrinkage (Stage 4) would lean on—but validation chose κ=0 on dense POG, so u_c=$\kappa/(f_c+\kappa)$=0, $\hat{\theta}$=Y, and the dress is left at rank 3971. The one dial that could rescue it is switched off here; it is switched *on* on the cold/sparse datasets, where it lifts such taxonomy-supported-but-unseen items by 2–5pp (`RESULTS.md`).

7.1 Why validate at NDCG@20 — and why not a "cutoff-free" loss

A fair question: if we *select* the weights on NDCG@20 and also *report* at cutoff 20, are we not just tuning to the number we are graded on? The honest answer has two parts, both measured (`results/selection_sensitivity.md`).

The result does not depend on the cutoff. Re-selecting the weights at NDCG@K for $K \in \{10, 20, 50, 100\}$ and re-measuring test Recall@20 gives *the same answer* across a tenfold range of cutoffs — a flat plateau (below). Selecting *directly* on Recall@20 is no better (electronic 0.486 vs. NDCG@20's 0.514). So $K = 20$ is not cherry-picked, and NDCG@20 is not "optimising the reported metric" — it is a smoother, better-conditioned surrogate at the same *operating point*.

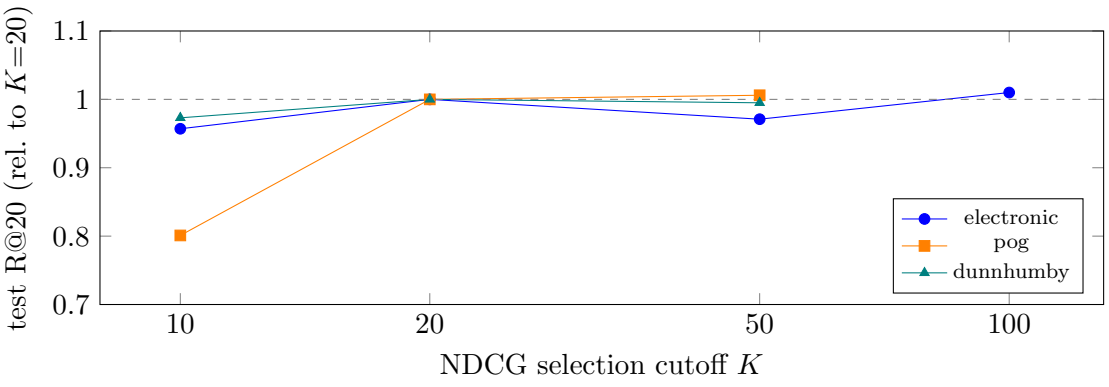


Figure 16. *Flat plateau.* Test Recall@20 (normalised to the K=20 run) is essentially unchanged whether weights are selected at NDCG@10, 20, 50 or 100. Cutoff-free objectives, by contrast, land anywhere from 0.03× to 0.94× depending on the dataset (table below).

A cutoff-free objective is worse, and unreliably so. The tempting "generic" alternatives all under-perform because they do not target the retrieval region a short recommendation slate actually uses:

selection objective	electronic	pog	dunnhumby
NDCG@20 (used)	**0.514**	**0.064**	**0.151**
Recall@20 (ERM on reported metric)	0.486	0.066	0.148
MRR (1/rank, cutoff-free)	0.327	**0.002**	0.142
softmax NLL (cutoff-free, proper score)	0.452	0.044	0.140

MRR weights rank-1 so heavily that moving a true item from rank 25 to 18 — exactly what Recall@20 rewards — barely registers; it is fine on dense collaborative data (dunnhumby) but *catastrophic* on sparse content data (pog, 0.002). The softmax NLL is a proper scoring rule for the *whole* score distribution, so it spends effort on calibration far below the slate and is consistently ∼7–31% below NDCG@20. **The lesson is the opposite of "do not tune to their metric": you should select at the operating point you deploy at** (a short slate), the exact cutoff inside a broad band does not matter, and no cutoff-free loss improves on it.

8 The same machine on a real POG query

We have read this one real POG_DENSE query signal by signal through the whole note; here is the entire thing in a single place—every signal's actual value side by side, and the four fusion stages that turn them into the filled bundle. Nothing is new or simplified: these are exactly the values dumped by `CASE_STUDY=1 BUNDLE_DS=pog_dense python -m bundlefuse.eval.fullrank` into `results/case_study_pog_dense.json`, the same ones quoted in the Examples above.

The query. The partial outfit is $S = \{$ a sequined chain shoulder bag, a pair of block-heel strappy sandals $\}$. Two members were held out and must be recovered: a pair of **drop earrings** and a rare **plus-size A-line dress**. Validation selected weights $w_{\texttt{cat}}$=1.0, $w_{\texttt{simp}}$=1.0, $w_{\texttt{softcat}}$=$w_{\texttt{bridge}}$=$w_{\texttt{pop}}$=0.25 (every other signal, including raw `cooc`, `ppmi` and `csvd`, pruned to weight 0), higher-order α=1, γ=0, and shrinkage κ=0. That weight pattern *is* the regime story of §7: on POG the **taxonomy** (`cat`) and the degree-normalised collaborative term (`simp`) carry the load; raw co-occurrence is redundant once `simp` is present.

			standardised signal z_k					**fusion**			
candidate (glossed)		f_c	`simp`	`cooc`	`cat`	`softcat`	`bridge`	Y	μ_0	$\hat{\theta}$	rank
drop earrings	**(held-out✓)**	686	124.1	162.7	1.83	3.94	3.42	**130.2**	8.21		**1**
pearl earrings (popular hub)		3077	21.9	19.1	1.83	4.25	1.98	28.4	8.68		—
same-brand bag (2nd bag!)		140	20.7	19.1	−1.01	0.71	2.29	22.2	2.05		—
black pointed heels		17	20.8	28.7	−0.39	−0.11	2.52	21.9	1.51		—
plus-size A-line dress	**(held-out, missed)**	2	−0.02	−0.02	1.71	−0.74	−0.58	1.52	2.06		3971

Figure 17. *One real* POG *partial bundle, every signal's actual number.* Each z_k is the signal's standardised score for that candidate (z-scored over all 31,217 items; the collaborative signals are very peaked—most items score ≈ 0—so a strong match sits many σ above the field, hence the large `simp`/`cooc` magnitudes). Y is the fused collaborative+higher-order score, μ_0 the taxonomy/content prior, $\hat{\theta}$ the routed final score; with κ=0 here $\hat{\theta}$=Y. Ranks are over the full catalogue with the seeds masked.

Reading the recovered item (earrings, rank 1). Everything agrees. The earrings co-occur heavily with bag+sandals outfits in training (`simp`= 124, far above the field), *and* "earrings" is a strongly complementary category to bags/shoes (`cat`= 1.83, `softcat`= 3.94). Because the two load-bearing weights are $w_{\texttt{simp}}$=$w_{\texttt{cat}}$=1, these add into $Y = 130.2$—an order of magnitude above every distractor—so the earrings land at rank 1. This is the *easy* completion: the typical co-purchased complement, backed by collaboration and taxonomy at once.

Reading the distractors—why the obvious-looking ones lose. The *pearl* earrings are a popularity hub (f_c=3077) with the same complementary category, so they score well (Y=28)—but they co-occur with *these particular* seeds far less than the true earrings (`simp` 22 vs. 124), so they stay well below. The **same-brand bag** is the clearest lesson: it co-occurs plausibly (`simp`= 20.7) and is even the same designer as the seed bag, yet it gets `cat`= −1.01—a *negative* category score from the slot self-exclusion term (§5.2), because an outfit does not take a *second bag of the same category*. Taxonomy actively vetoes it.

Reading the missed item (the rare dress, rank 3971)—and what it reveals. The dress is genuinely category-appropriate (`cat`= 1.71, a positive complement), but it was seen in only f_c=2 training outfits and *never* with anything resembling these seeds, so every collaborative signal is silent on it (`simp`≈ `cooc` ≈ 0). With the collaborative term contributing nothing, its fused score is just the lone `cat` vote, $Y = 1.52$—invisible against items that have *both* kinds of evidence. Notice that the prior *does* like it slightly more ($\mu_0 = 2.06 > Y$): the taxonomy-only signal is exactly the kind of evidence Stage 4 shrinkage is meant to lean on for low-exposure items. But here κ=0—validation turned shrinkage *off*, because POG_DENSE has almost no cold items and buying unearned shrinkage would only add noise (the honest finding of §7 and `results/matrix_zeroparam.csv`). So on this dense dataset the rare dress is left behind; it is precisely on the *cold/sparse* datasets, where $\kappa > 0$ is selected, that this same mechanism pulls such taxonomy-supported-but-unseen items up into the slate (the +2–5pp cold-bucket gain reported in `RESULTS.md`). One honest query thus shows both what the method gets right and the exact regime where its one real dial earns its place.

9 The ledger: what is learned vs. fixed

Fixed inputs (not learned)	embeddings (§2.2, shipped with the dataset); category/artist/album labels; the bundle incidence matrix B.
Deterministic constructions	C, PPMI, PPMI-SVD fingerprints, W_{pair}, category/soft-cluster PMI, disentangled + bridged content. All closed-form functions of the fixed inputs—*zero trainable parameters.*
Selected on validation (the only "learning")	the per-signal weights w_k (grid $\{0, 0.25, 0.5, 1, 1.5\}$); the higher-order mixers α, γ; the shrinkage dial κ; the prior scale (computed, then frozen). A few dozen scalar choices total, by grid search—no gradients, no neural weights.
Off by default	the $\leq$ 0.30M-parameter contrastive set-encoder (`ENCODER=0`).

In one paragraph. BundleFuse turns the bundle incidence matrix and a set of fixed content embeddings into about eleven standardised "opinions" per candidate—raw and chance-corrected co-occurrence, a co-occurrence SVD fingerprint, size-aware pairwise and two-hop scores, two small set-level corrections, category and soft-category complementarity PMIs, metadata homophily, and category-disentangled / graph-bridged content similarity. It blends them with validation-picked weights into a collaborative score Y and a cold-start prior μ_0, then routes the two per candidate by $\hat{\theta} = (1 - u_c)Y + u_c\mu_0$ with $u_c = \kappa/(f_c + \kappa)$, so frequent items are judged by co-occurrence and rare/cold items by what they *are*. The scoring core trains nothing; the embeddings are inputs, and the handful of dials are chosen by grid search on held-out bundles.